Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · WEEK 2

Operating System Structure

The services an operating system offers, the system calls that reach them, and the ways a kernel can be structured.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Identify the services an operating system provides.
2. Show how system calls are used to provide those services.
3. Compare monolithic, layered, microkernel, modular and hybrid designs.
4. Describe how an operating system boots.
5. Use tools to monitor operating-system performance.

THIS WEEK

Topic 2

Week 1 said what an operating system is. This week looks at it from three sides: its users, its programmers and its designers.

DISCUSSION

How does printf reach the screen?

Your program cannot touch the display hardware. So who does, and how is it asked?

PART 1

Operating-system services

An environment in which programs run, and services for those programs and their users.

Week 2

Services that help the user

User interface

A command line, a graphical interface or a touch screen.

Program execution

Load a program into memory, run it, and end it normally or with an error.

I/O operations

Programs cannot control devices directly, so the system does I/O for them.

File-system manipulation

Create, read, write, delete and search files and directories, and manage permissions.

Communications

Processes exchange information on one computer or across a network.

Error detection

Detect errors in hardware, I/O and programs, and take the right action for each.

Services that keep the system efficient



Resource allocation

When several processes run at once, CPU cycles, main memory, file storage and devices must be shared out among them.



Logging

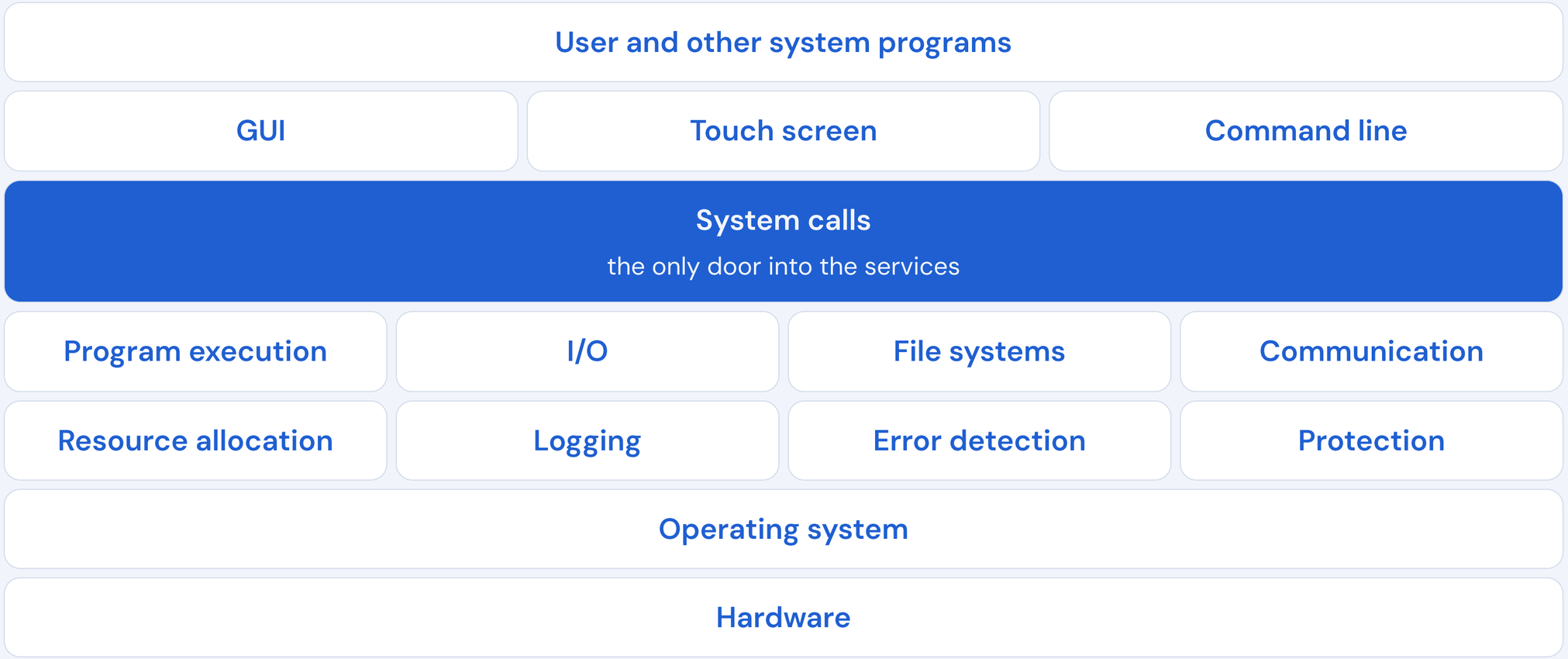
Keep track of which programs use how much of which resources, for accounting or for tuning.



Protection and security

Control all access to system resources, and defend the system against outsiders, starting with authentication.

A view of operating-system services



PART 2

User and operating-system interface

Three ways for a person to tell the system what to do.

The command interpreter

A *shell* reads a command from the user and executes it.

Some commands are built into the shell. Most are just the names of programs: the shell finds the file, loads it and runs it.

That second design means a new command needs no change to the shell. We only add a file.

```
$ pwd
/home/student
$ ls -l notes.txt
-rw-r--r-- 1 student gic 482 notes.txt
$ rm notes.txt
$ echo $SHELL
/bin/bash
```

Here ls and rm are programs in /bin. The shell did not need to know what they do.

Graphical and touch-screen interfaces

GUI

A desktop metaphor: a mouse, windows, menus and icons. Icons stand for files, programs and actions. Most systems offer both a GUI and a command line.

Touch screen

A mouse is not practical on a phone. Actions are gestures: taps, swipes and pinches. The keyboard is on the screen.

Which one?

Mostly personal preference. Administrators and power users often choose the command line: it is faster for repeated tasks and can be scripted.

PART 3

System calls

The programming interface to the services of the operating system.

What a system call is

A request from a program to the kernel, usually made through an *API* and not directly.

Written in a high-level language

System calls are available as functions in C and C++. A few low-level tasks need assembly.

Reached through an API

The Windows API, the POSIX API for UNIX, Linux and macOS, and the Java API for the Java virtual machine.

Why an API

Portability: a program written for the API runs on any system that supports it. And the real system calls are more detailed and harder to use.

A simple task needs many system calls

```
int in = open("a.txt", O_RDONLY);
int out = open("b.txt", O_WRONLY | O_CREAT, 0644);

while ((n = read(in, buf, SIZE)) > 0)
    write(out, buf, n);

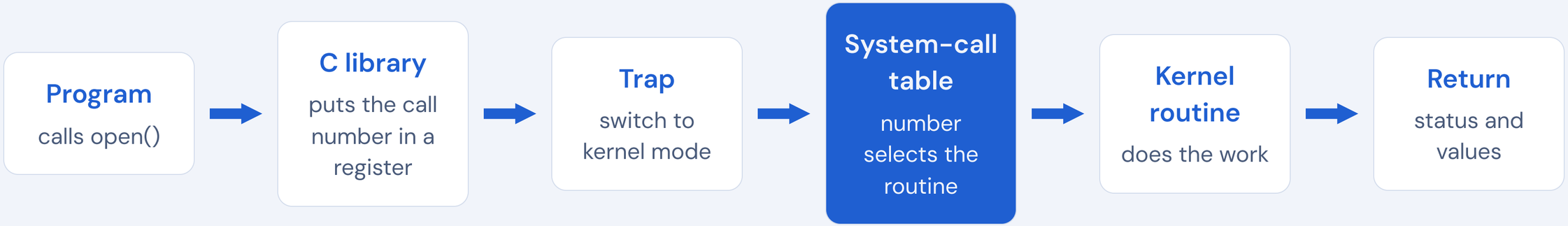
close(in);
close(out);
```

Copying one file to another.

Open the input. Create the output. Loop: read a block, write a block. Close both.

Each of these is a system call, and each can fail: the input may not exist, the disk may be full.

How a system call is carried out



Each system call has a number. The system-call interface keeps a table indexed by these numbers. The caller needs to know nothing about the implementation: only the API and what comes back.

Passing parameters to the kernel

01

In registers

The simplest way. But there may be more parameters than registers.

02

In a block

Parameters are stored in a block or table in memory, and the address of the block is passed in a register. Linux uses this when there are more than five parameters.

03

On the stack

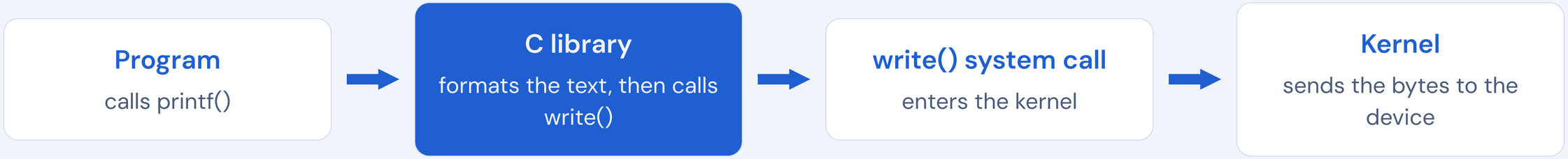
The program pushes the parameters onto the stack and the operating system pops them off.

The block and stack methods do not limit the number or length of the parameters.

Types of system calls

Category	What they do	UNIX examples
Process control	Create, end, load, execute and wait for processes; allocate memory	fork(), exit(), wait()
File management	Create, delete, open, close, read and write files; get attributes	open(), read(), write(), close()
Device management	Request and release a device; read and write it	ioctl(), read(), write()
Information maintenance	Get or set the time, date and system data	getpid(), alarm(), sleep()
Communications	Create connections; send and receive messages; share memory	pipe(), shm_open(), mmap()
Protection	Get and set permissions; allow or deny access	chmod(), umask(), chown()

The standard C library



This answers the opening question. printf() is a library function in user mode. It ends in a write() system call, and only the kernel touches the display.

Single-tasking and multitasking systems

ONE PROGRAM

Single-tasking

A microcontroller board such as Arduino has no operating system in the usual sense. A boot loader loads one program, called a sketch, into memory and runs it. The program has the whole machine.

MANY PROCESSES

Multitasking

On FreeBSD, the shell runs fork() to create a process and exec() to load a program into it. The shell then waits for the process, or carries on. A process ends with exit(): code 0 means no error.

PART 4

System services, linkers and loaders

The programs around the kernel, and how our own code becomes a running program.

Week 2

System services

File management

Create, delete, copy, rename and list files and directories.

Status information

Date, time, memory and disk use, logs, and performance data.

File modification

Text editors, and commands that search and transform text.

Programming support

Compilers, assemblers, debuggers and interpreters.

Loading and execution

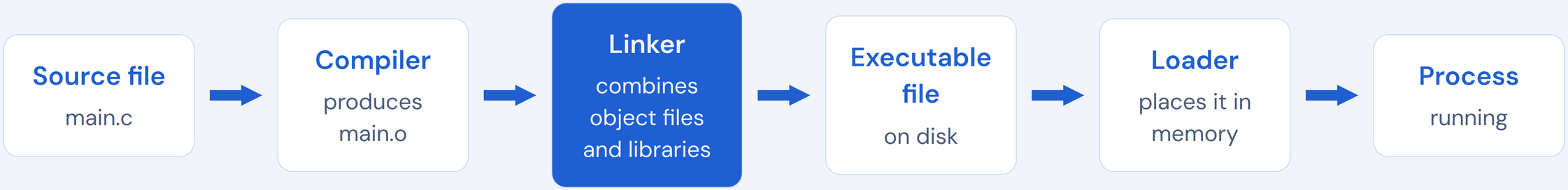
Loaders, linkers and debugging systems.

Background services

Daemons started at boot: network, printing, scheduling of jobs.

Most users see the operating system through these programs, not through the system calls beneath them.

From source code to a running process



Relocatable object file

Compiled code that can be loaded at any address.

Relocation

Final addresses are assigned to the parts of the program, and the code is adjusted to match.

Dynamic linking

Libraries are not copied into the executable. They are loaded when needed and shared by every program that uses them.

Why applications are operating-system specific



The problem

Each operating system has its own system calls, its own executable file format, and runs on CPUs with their own instruction sets.



Three ways around it

Write in an interpreted language such as Python. Write for a virtual machine such as Java. Or write in a standard language and API, and compile once for each system.



API and ABI

An API works at the source level. An application binary interface is its equivalent for compiled code: it fixes how binaries fit a given system and CPU.

PART 5

Design and structure

No single right answer, but a few approaches that have worked.

Design goals

FOR THOSE WHO USE IT

User goals

Convenient to use, easy to learn, reliable, safe and fast.

FOR THOSE WHO BUILD IT

System goals

Easy to design, implement and maintain. Flexible, reliable, free of errors and efficient.

The goals depend on the hardware and on the type of system: a phone, a server and a car's controller want different things.

Policy and mechanism

Policy: what will be done. *Mechanism*: how to do it.

Example: the timer

The timer is a mechanism for protecting the CPU. How long each user may run before the timer fires is a policy decision.

Why separate them

Policies change with place and time. If the mechanism is independent, a new policy means changing a parameter, not rewriting the system.

Implementation

A mix of languages

The lowest levels in assembly, the kernel in C, system programs in C and C++, and scripts in languages such as Python and shell.

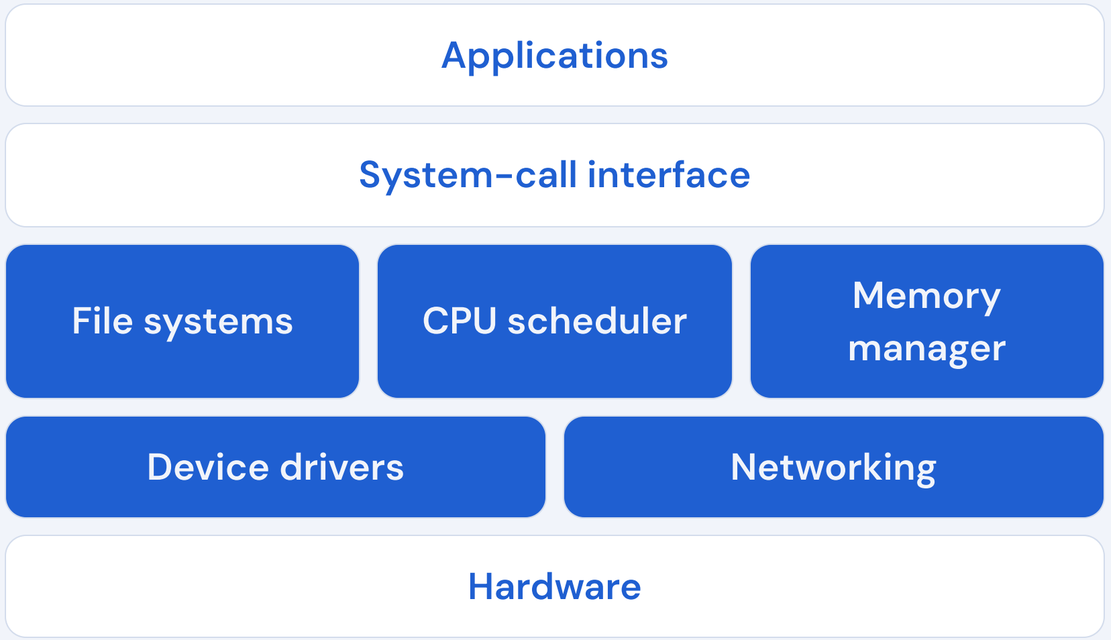
Why a high-level language

The code is written faster, is more compact, and is easier to understand and debug. It is also easier to port to other hardware.

The cost

Possibly less speed and more storage. Better algorithms and data structures matter far more than hand-written assembly.

Monolithic structure

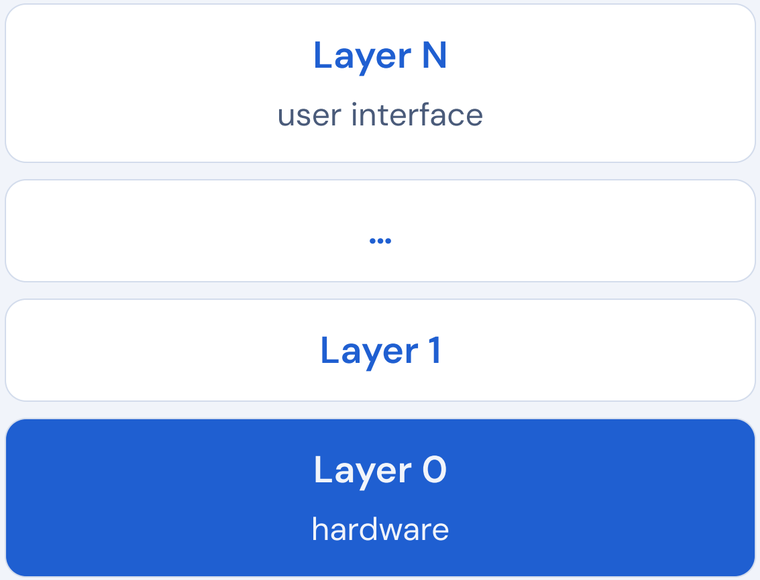


The whole kernel is one program in one address space.

The original UNIX had two parts: the kernel and the system programs. Everything between the system-call interface and the hardware was the kernel.

Fast, because parts call each other directly. Hard to implement and extend, because everything can affect everything.

The layered approach



The idea

Each layer uses only the functions and services of the layers below it.

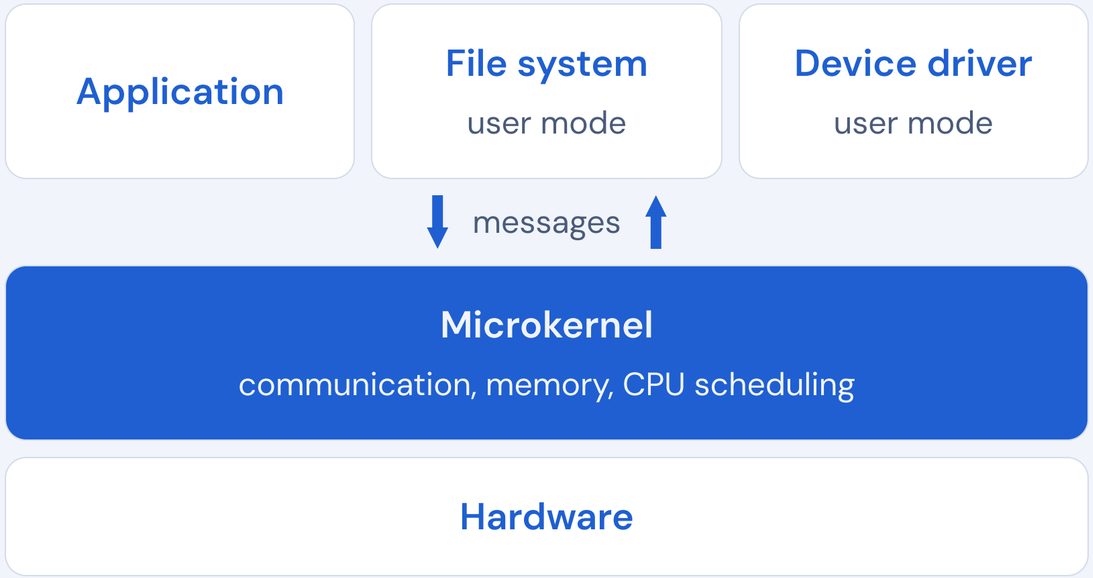
For

Simple to build and debug: layer 1 can be tested with only the hardware beneath it, then layer 2, and so on.

Against

Hard to define the layers, and each layer a request passes through adds overhead.

Microkernels



Move as much as possible out of the kernel into user space.

Services talk to each other by message passing. Mach is the best-known example; the Darwin kernel of macOS is partly based on it.

Easier to extend and to port, more reliable and more secure, since less code runs in kernel mode.

The cost is the overhead of passing messages between user space and kernel space.

Modules and hybrid systems



Loadable kernel modules

The kernel has a set of core components and links in more services as modules, at boot or while running. Each module talks to the others through known interfaces. Linux and Solaris work this way.

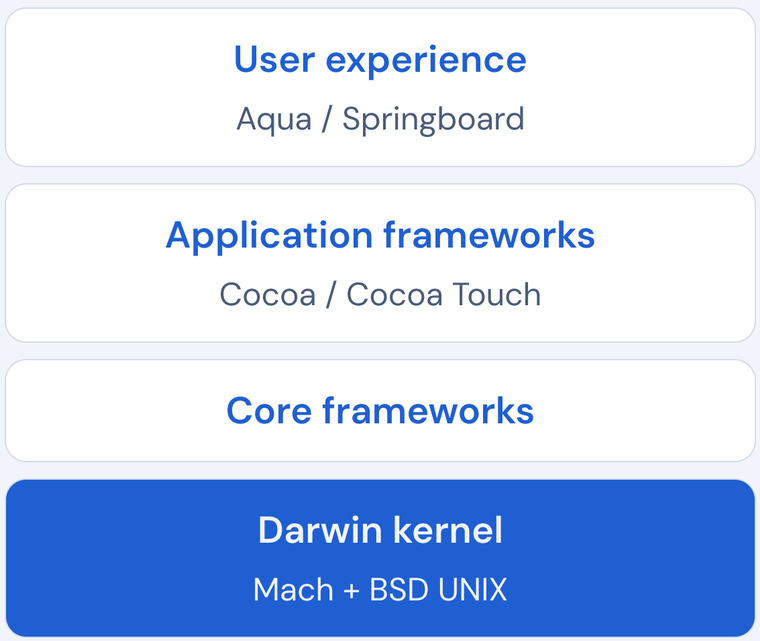


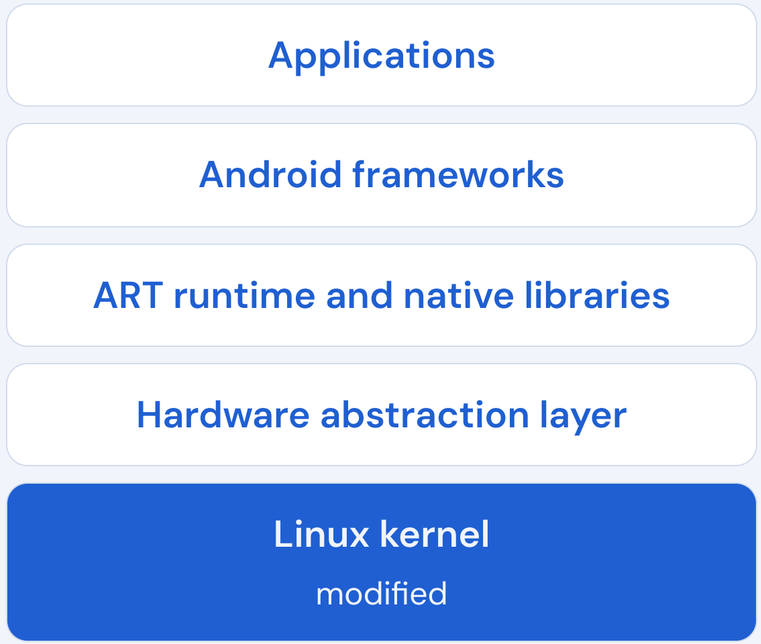
Hybrid systems

Most real systems combine approaches. Linux is monolithic for speed and modular for flexibility. Windows is mostly monolithic, with some microkernel ideas for its subsystems.

Modules give the flexibility of layers without their overhead: any module can call any other.

macOS, iOS and Android





Both are layered stacks on a hybrid kernel.

Apple's systems are closed apart from Darwin. Android is open source, developed by the Open Handset Alliance led by Google.

Android apps are written in Java and compiled ahead of time for the Android runtime.

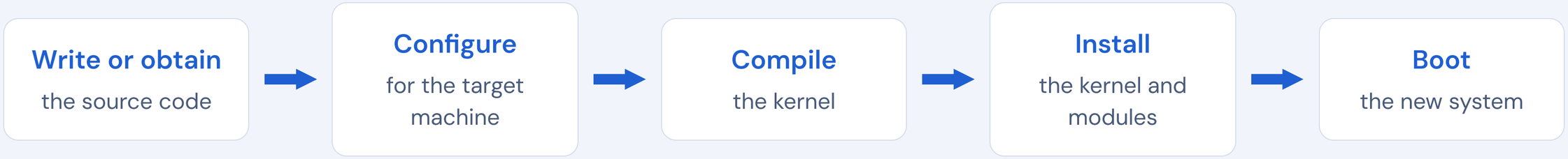
PART 6

Booting and debugging

Getting the kernel running, and finding out what it is doing.

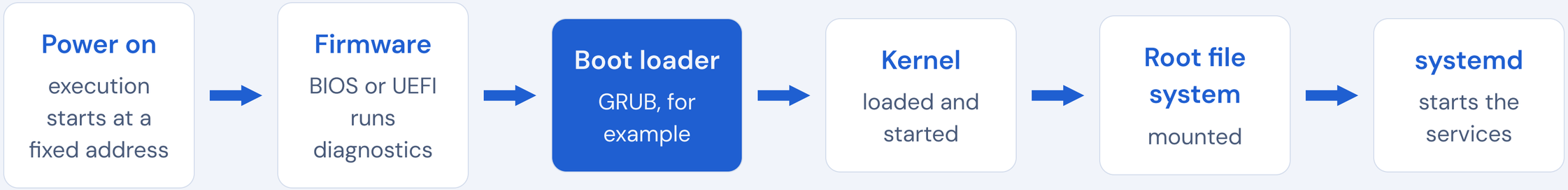
Week 2

Building an operating system



For Linux: download the source, run the configuration step, compile the kernel and its modules, install them, and restart. A general-purpose system is usually built once for many machines and then tuned by its configuration.

System boot



The boot loader lets us choose a kernel and pass it parameters. UEFI replaces the older BIOS: it is faster and knows about larger disks.

Operating-system debugging



Log files

The system writes error information to log files as it runs.



Core dump

When an application fails, a copy of the process's memory is saved for analysis.



Crash dump

When the kernel itself fails, the kernel's memory is saved.

A failure in a process is an inconvenience. A failure in the kernel is called a crash.

Performance monitoring and tracing

Scope	Counters	Tracing
One process	ps, top	strace: the system calls a process makes gdb: a source-level debugger
The whole system	vmstat, netstat, iostat	perf: Linux performance tools tcpdump: network packets

Counters

Numbers the system keeps as it runs, such as system calls made or packets sent. On Linux many are read from the /proc file system.

Tracing

Collects data for a specific event, such as the steps of one system call.

BCC and eBPF

A toolkit for tracing a live Linux system safely and with little overhead.

Key takeaways

01

Services for users and for the system

From running programs and doing I/O to allocating resources and protecting them.

02

System calls are the door

Programs reach the kernel through system calls, usually by way of an API such as POSIX.

03

Policy apart from mechanism

Separating what is done from how it is done keeps a system flexible.

04

Structures are hybrids

Monolithic, layered, microkernel and modular ideas are combined in real systems.

Next session

COMING UP

Processes

What a process is, how processes are created, scheduled and ended, and how they communicate.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 2.